

PHYSICAL ROBOT INTERFACE

AND SIM-TO-HARDWARE BASELINE

Autonomous Semantic Navigation Robot

Jan Cuesta

ASU TurtleBot 4 Standard | ROS 2 Jazzy

Version 0.9 – Development baseline draft – 24 September 2026

Purpose. Turn the professor-confirmed robot identity into a practical interface contract for V1 simulation and V2 hardware transfer, while clearly separating confirmed facts, manufacturer nominal data, and facts that must be measured on the ASU unit.

Controlled status: sufficient to proceed with architecture and implementation. Complete the on-unit capture before physical

autonomous motion.

DOCUMENT CONTROL		Lean implementation record
Field	Value	
Document owner	Jan Cuesta	
Project	Autonomous Semantic Navigation Robot	
Document identifier	ASN-PHY-IF-S2H	
Revision / status	0.9 / development baseline draft	
Related baselines	V1 ConOps and Scope v0.9; V1 Requirements, Traceability, and Risk v0.9	
Physical target	ASU TurtleBot 4 Standard	
Tailoring principle	Record only facts and checks that affect software design, safe integration, reproducibility, or sim-to-real transfer.	

Revision and approval

Version	Date	Change	Status
0.9	24 Sep 2026	Initial physical platform, interface, safety, mismatch, and on-unit verification baseline.	Review
1.0	Pending	Replace on-unit TBDs with captured evidence and approve the V2 interface contract.	Frozen before V2 motion

Role	Name / approval	Date
Document owner / developer	Jan Cuesta: _____	_____
Lab owner / supervisor	_____	_____

Step 3 result. The physical platform is no longer generic: use TurtleBot 4 Standard conventions as the V1-to-V2 target. Do not delay V1 coding for details that can only be captured from the ASU unit.

1. Evidence hierarchy

Status	Source	Use
Confirmed	Professor Vivek Thangavelu reply, 23 Sep 2026	Binding project fact unless ASU identifies a different unit.
Manufacturer nominal	TurtleBot 4 / Create 3 official documentation	Design input; verify configuration-sensitive values on the unit.
On-unit verified	Command output, photo, measurement, or controlled test	Binding V2 interface fact after capture.
Project decision	V1/V2 baseline documents	Conservative software limit or abstraction; may intentionally be below hardware capability.
Open	Not yet observed or approved	Must not be represented as confirmed.

01 / LOCKED PLATFORM IDENTITY

What is known now

2. Professor-confirmed facts

The ASU platform is a **TurtleBot 4 Standard**. It runs **Ubuntu 24.04** and **ROS 2 Jazzy** on an onboard **Raspberry Pi 4**. Professor Thangavelu directed the project to use the TurtleBot 4 documentation for the remaining specification and interface details.

This confirmation aligns the physical target with the V1 software baseline. It does not yet confirm checkout terms, the exact robot asset, installed package revisions, Create 3 firmware, namespace, network configuration, or whether the ASU unit has been modified.

3. Manufacturer nominal configuration

Item	Nominal value	Project treatment
Base / drive	iRobot Create 3; differential drive	V1 and V2 kinematic class is locked.
Overall size	342 x 339 x 351 mm	Use for packaging; navigation uses the official footprint below.
Navigation footprint	Eight-point polygon at 0.189 m cardinal radius	Initial V1/V2 Nav2 footprint; verify installed config.
Wheel diameter	72 mm	Use nominally; verify URDF and unit before calibration work.
Mass	3.945 kg	Simulation dynamics input, not an acceptance threshold.
Onboard computer	Raspberry Pi 4B, 4 GB	Confirmed family; RAM/configuration nominal until inspected.
LiDAR	RPLIDAR A1M8, 360 degrees, 12 m stated range	Primary V1/V2 planar navigation sensor.
Camera	OAK-D-Pro	Present for future V3; not required for V1 acceptance.
Maximum velocity	0.31 m/s safe; 0.46 m/s with safety fully disabled; 1.90 rad/s angular	Hardware capability only. Project limits remain lower.
Battery / runtime	26 Wh; 2.5–4 h load-dependent	Planning value; capture battery health and observed runtime later.
Odometry source	Create 3 fused wheel encoders, IMU, and ground optical flow	‘/odom’ is the V2 odometry input; raw sources remain diagnostic.

No safety-feature bypass. The project shall retain Create 3 safety behavior. The 0.46 m/s figure is not authorized for this project. V1/V2 navigation remains limited to 0.25 m/s linear and 0.75 rad/s angular unless the requirements baseline is deliberately revised.

4. Configuration facts still open

- ASU asset/serial number, physical condition, modifications, charger/controller/cables, and access rules.
- Exact Raspberry Pi image, package versions, Create 3 firmware, RMW implementation, namespace, ROS domain, and discovery mode.
- Installed topic names, message types, QoS, rates, TF tree, LiDAR alignment, wheel separation, and effective command timeout.
- Lab test area, floor/lighting/network conditions, approved operating hours, and supervisor safety constraints.

02 / SOFTWARE INTERFACE CONTRACT

Stable names outside, adapters at the boundary

5. Project-facing ROS 2 interfaces

The project shall keep application code independent of simulator- or robot-specific wiring. Launch files and adapters own remapping, message conversion, namespace, and QoS differences.

Purpose	Project interface	Type / direction	Status
Navigation goal	/navigate_to_pose	nav2_msgs/action/NavigateToPose	Stable project API
Velocity command	/cmd_vel	geometry_msgs/msg/TwistStamped, outbound on Jazzy	Candidate; verify unit
Unstamped compatibility	/cmd_vel_unstamped	geometry_msgs/msg/Twist, outbound	Adapter/fallback only
Planar scan	/scan	sensor_msgs/msg/LaserScan, inbound	Manufacturer convention
Fused odometry	/odom	nav_msgs/msg/Odometry, inbound	Manufacturer convention
IMU	/imu	sensor_msgs/msg/Imu, inbound	Manufacturer convention
Transforms	/tf, /tf_static	tf2_msgs/msg/TFMessage, inbound/outbound	Standard convention
Battery	/battery_state	sensor_msgs/msg/BatteryState, inbound	V2 health input
Hazards	/hazard_detection	irobot_create_msgs/msg/HazardDetectionVector	Enable in republisher for V2
Wheel state	/wheel_status	irobot_create_msgs/msg/WheelStatus	Verify E-stop / diagnostics
Stop state	/stop_status	irobot_create_msgs/msg/StopStatus	Optional evidence; low rate
Slip state	/slip_status	irobot_create_msgs/msg/SlipStatus	Recommended diagnostic
Motor E-stop	/e_stop	irobot_create_msgs/srv/EStop	Required V2 control
Base power-off	/robot_power	irobot_create_msgs/srv/RobotPower	Shutdown only; not motion stop

Jazzy command detail. The current TurtleBot 4 Jazzy Nav2 configuration enables stamped velocity commands. The Create 3 republisher supports both stamped and unstamped inputs and may remap the public names. The ASU unit's actual 'ros2 topic info -verbose' output is authoritative. Only one command path may own the base at a time.

6. Frame contract

Frame / transform	Owner	Rule
'map' to 'odom'	SLAM Toolbox or AMCL	Exactly one localization source publishes it.
'odom' to 'base_link' / 'base_footprint'	Create 3 in V2; simulator base in V1	Fused physical odometry in V2; no ground-truth substitution.
Robot static tree	'robot_state_publisher'	Use TurtleBot 4 Standard description or equivalent matched V1 model.
LiDAR	'rplidar_link'	'/scan.header.frame_id' must connect to 'base_link'.
Camera	OAK-D frame family	Preserve for V3; no V1 navigation dependency.

7. Deployment split

The Raspberry Pi runs robot bringup, Create 3 republisher, sensor drivers, robot description, and safety/health endpoints. A development PC may run RViz, mapping, localization, Nav2, logging, and project nodes during early integration. Final placement is selected by measured CPU, memory, network latency, and reliability; the node APIs do not change.

03 / STOP AND SAFETY CONTRACT

Stop motion first; preserve evidence when possible

8. Three distinct stop paths

Control	Immediate effect	Resulting state	Reset
Normal cancel	Cancel goal and command zero velocity	Stationary, system Ready	New goal allowed
Manual Safety Stop	Cancel goal, command zero, latch project motion inhibit	Stationary, diagnostics alive	Explicit project reset after checks
Emergency Stop	Call Create 3 <code>/e_stop</code> with <code>'e_stop_on: true'</code> ; wheel power disabled	Motion impossible; compute and evidence retained when practical	Explicit <code>/e_stop</code> release plus Ready checks
Power shutdown	Orderly software shutdown, then base power-off when required	System Off	Full restart

Emergency behavior. For the TurtleBot 4, the emergency objective is immediate motor-power inhibition, not abruptly removing Raspberry Pi power. Abrupt compute power loss can corrupt storage and removes diagnostics. `/robot_power` is not a substitute for `/e_stop`.

9. Independence and limitations

The Create 3 E-stop service disables wheel power and rejects velocity commands, but it still depends on the ROS communication path used to issue it. Therefore it is a functional motor E-stop, not evidence of a certified or independent hardware safety circuit. Before V2 autonomous motion:

- keep the operator within immediate reach of the robot and the physical power control;
- prove `/e_stop` locally and from the operator station at very low speed in an open area;
- verify disabled wheels through `'wheel_status'`, stopped odometry, and direct observation;
- agree with the lab owner whether an additional independent motor-power cutoff is required;
- retain bump, cliff, wheel-drop, backup-limit, and safe-speed protections.

10. Command watchdog

The V1 project watchdog remains: if the selected command stream is stale for 0.50 s, publish zero within 0.10 s, inhibit further nonzero output, and verify stopped motion within 1.0 s. The Create 3 also has an internal velocity-command timeout; its installed value and measured stop response shall be captured on the ASU unit. The two layers are complementary.

11. Clearance interpretation

The current 0.20 m minimum obstacle separation is a provisional navigation target measured from the configured robot footprint to sensed obstacle geometry during nominal motion. Physical bumper contact is always a collision, even if the path planner believed clearance was positive. Create 3 proximity and hazard signals supplement the LiDAR costmap but do not replace it.

04 / SIM-TO-HARDWARE MISMATCHES

Design them out before V2

12. Controlled mismatch register

ID	Area	Expected mismatch	Required treatment / evidence
S2H-01	Geometry	Simplified V1 body versus TB4 Standard shell and footprint	Use official eight-point footprint initially; run inflation and doorway checks; photograph/measure unit.
S2H-02	Kinematics	Ideal wheel model versus real calibration, compliance, and caster behavior	Load vendor URDF/controller values; measure straight/rotation error; change only with evidence.
S2H-03	Odometry	Simulated wheel odometry versus Create 3 fused encoders, IMU, and optical flow	Use ‘/odom’; test drift, covariance, slip, and floor dependence.
S2H-04	LiDAR	Ideal scans versus noise, occlusion, reflective surfaces, and mounting error	Match rate/FOV/range where practical; record bags; verify frame and scan orientation.
S2H-05	Timing	Deterministic local simulation versus DDS, Wi-Fi, USB, and SBC load	Time-sync hosts; measure rates, age, command latency, packet loss, and throttling.
S2H-06	Command API	Simulator bridge may accept unstamped Twist; Jazzy TB4 navigation uses stamped Twist	Central adapter/remap; verify type and only one active command owner.
S2H-07	Safety	Simulated collision/stop versus bump, cliff, wheel-drop, backup limit, and motor E-stop	Preserve safety features; bench-test each applicable stop signal before autonomous runs.
S2H-08	Dynamics	Approximate mass/friction/acceleration versus floor- and payload-dependent motion	Begin at 0.10 m/s; tune acceleration/controller before increasing to project limits.
S2H-09	Localization	Perfectly known simulated start versus approximate physical initial pose and sensor ambiguity	Reuse saved map workflow; test AMCL convergence and recovery at S1–S3.
S2H-10	Environment	Programmed geometry versus real clutter, moving people, and changing doors	Static map contains permanent geometry; live costmap handles temporary/moving objects.
S2H-11	Compute	Desktop/GPU simulation versus Raspberry Pi resource limits	Profile placement; disable unnecessary streams; offload visualization/heavy perception.
S2H-12	Camera	V1 excludes RGB-D; physical Standard includes OAK-D-Pro	Keep disabled for V1/V2 navigation baseline; integrate intentionally in V3.

Companion register. The editable CSV contains these rows plus owner, closure criterion, status, and evidence fields. Update the CSV during integration; revise this PDF only when the contract changes materially.

13. Immediate V1 implementation consequences

- Model a TurtleBot 4 Standard-class differential-drive footprint and sensor placement; visual mesh fidelity is optional.
- Expose stable project topics/actions and place Isaac-specific bridges behind launch/configuration.

- Use stamped velocity commands internally for Jazzy; provide exactly one explicit unstamped adapter if required.
- Keep navigation limits at 0.25 m/s and 0.75 rad/s, below manufacturer capability.
- Simulate LiDAR and wheel-derived odometry for V1; do not consume ground truth in SLAM, localization, or Nav2.
- Implement stop arbitration, command freshness monitoring, health status, and evidence capture so the same project node can transfer to V2.

05 / ON-UNIT CAPTURE

One focused session before V2 development

14. Capture package

Store one dated hardware-baseline evidence folder containing:

- photos of the robot, labels, sensors, charger, controller, ports, and any modification;
- ‘uname’, Ubuntu release, ROS distribution, package versions, image version, and Create 3 firmware;
- namespace, ROS domain, RMW, discovery mode, hostnames/IP strategy, and time synchronization;
- topic/service/action lists with types; verbose details for project-critical interfaces;
- TF frame graph, URDF/xacro, Nav2 footprint, sensor frames, parameters, and launch/config files;
- short rosbag while stationary, rolling slowly, rotating, and activating a controlled hazard/stop;
- measured dimensions needed for navigation plus test-floor and network notes.

15. Minimal command checklist

Run commands only after following ASU access and network instructions. Save outputs rather than manually copying values.

Capture	Command / method
OS and ROS	‘lsb_release -a’; ‘uname -a’; ‘printenv ROS_DISTRO’; package query
Discovery	‘ros2 topic list --no-daemon --spin-time 10 -t’
Critical topics	‘ros2 topic info --verbose’ for command, scan, odom, IMU, hazards, wheel status
Services/actions	‘ros2 service list -t’; ‘ros2 action list -t’
Rates	‘ros2 topic hz’ for scan, odom, IMU, TF, command stream
Frames	‘ros2 run tf2_tools view_frames’ and RViz inspection
Parameters	‘ros2 param list’; targeted ‘ros2 param dump’ for robot, sensors, Nav2
Evidence	rosbag of selected topics plus console and configuration snapshot

16. Safe first-motion sequence

1. Inspect unit, charge battery, clear a level test area, and identify the operator stop controls.
2. Start with wheels unable to drive the robot into anything; verify topic discovery, sensor freshness, TF, and command ownership.
3. Engage ‘/e_stop’; confirm wheel power disabled and nonzero commands do not move the robot. Release only after an explicit check.
4. At no more than 0.10 m/s, test forward, reverse, rotation, normal cancel, command loss, Manual Safety Stop, and Emergency Stop.
5. Record stop time/distance, odometry, warnings, hazards, CPU/memory/temperature, and any network loss.
6. Only then proceed to low-speed mapping and localization; increase speed within project limits after stable evidence.

No autonomous motion on first contact. The first hardware session is for identification, interfaces, sensors, stop controls, and controlled low-speed motion. Autonomous Nav2 trials begin only after the interface and stop checks pass.

06 / TRANSITION GATE

What must be true before V2 autonomy

17. V1-to-V2 readiness gate

Gate ID	Condition	Evidence
PHY-01	Exact ASU asset and installed hardware/software configuration captured.	Photos + inventory
PHY-02	Project PC and robot communicate reliably with matching ROS/RMW/discovery settings.	Logged discovery test
PHY-03	Command, scan, odom, IMU, TF, battery, hazard, wheel-state, and E-stop interfaces verified.	CLI output + bag
PHY-04	Only one velocity source owns the base in every mode; watchdog and zero-command behavior pass.	Controlled test
PHY-05	Normal Cancel, Manual Safety Stop, and Emergency Stop are distinguishable and repeatable.	Stop test record
PHY-06	Effective footprint, sensor frames, wheel geometry source, and speed/acceleration limits are recorded.	Config snapshot
PHY-07	Low-speed motion, odometry, scan orientation, TF continuity, and time synchronization pass.	Bag + analysis
PHY-08	Lab owner approves the test area, operating method, and any required additional cutoff.	Approval note
PHY-09	V1 acceptance evidence is complete enough to justify transferring the stack.	V1 release record

Implementation sequencing. Step 3 is complete enough at Version 0.9. Next: freeze software versions, define node/interface architecture, create the repository skeleton, and begin V1 implementation. The remaining Step 3 items are filled in during the first authorized robot session.

18. Change rules

- Manufacturer values may be corrected from official documentation without changing requirements unless behavior or risk changes.
- An on-unit observation overrides a nominal value for the ASU asset and must include evidence.
- Changes to safety behavior, command ownership, frame semantics, or the public project API require an impact review across requirements, architecture, tests, and configuration.
- Tuning values remain in version-controlled YAML; do not embed environment-specific names in application code.

19. References

1. Professor Vivek Thangavelu, email reply to Jan Cuesta, 23 Sep 2026. Confirms TurtleBot 4 Standard, Ubuntu 24.04, ROS 2 Jazzy, Raspberry Pi 4, and use of TurtleBot 4 documentation.
2. TurtleBot 4 User Manual, Features and Hardware Specifications, v2.0.1. <https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html>
3. TurtleBot 4 User Manual, Software Overview and Jazzy image changelog. <https://turtlebot.github.io/turtlebot4-user-manual/software/overview.html>
4. TurtleBot 4 Jazzy Nav2 configuration, ‘turtlebot4_navigation/config/nav2.yaml’. https://github.com/turtlebot/turtlebot4/blob/jazzy/turtlebot4_navigation/config/nav2.yaml
5. TurtleBot 4 User Manual, Create 3 and sensor interfaces. <https://turtlebot.github.io/turtlebot4-user-manual/software/create3.html>
6. iRobot Create 3 Docs, ROS 2 API, odometry, safety, moving the robot, and hazards. https://iroboteducation.github.io/create3_docs/api/ros2/
7. iRobot Create 3 Jazzy republisher parameters. https://github.com/iRobotEducation/create3_examples/blob/jazzy/create3_republisher/bringup/params.yaml